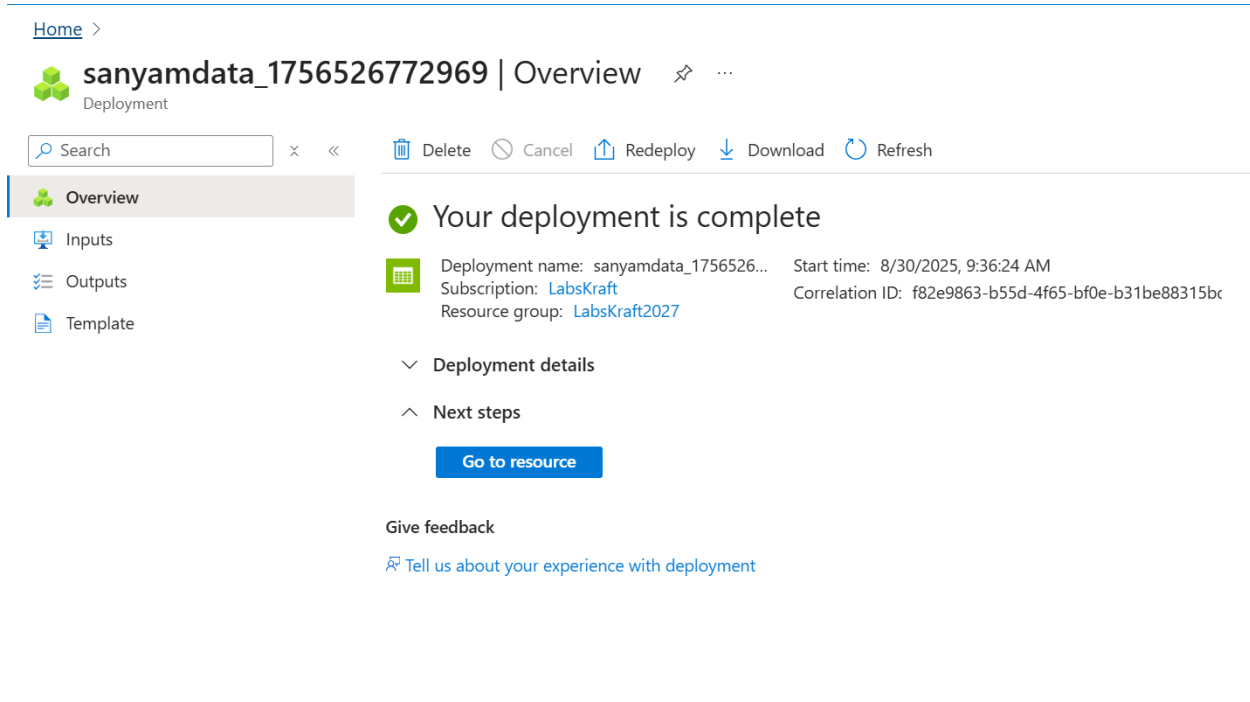


SANYAM ARORA-FINAL CAPSTONE PROJECT-REPORT

Step-1 : I have created a storage account named 'sanyamdata', with Azure Data Lake Storage Gen2 enabled as the primary service, and hierarchical namespace support activated.



Home > **sanyamdata_1756526772969** | Overview

Deployment

Search

Delete Cancel Redeploy Download Refresh

Overview

- Inputs
- Outputs
- Template

✓ Your deployment is complete

Deployment name: sanyamdata_1756526... Start time: 8/30/2025, 9:36:24 AM
Subscription: [LabsKraft](#) Correlation ID: f82e9863-b55d-4f65-bf0e-b31be88315bc
Resource group: [LabsKraft2027](#)

Deployment details

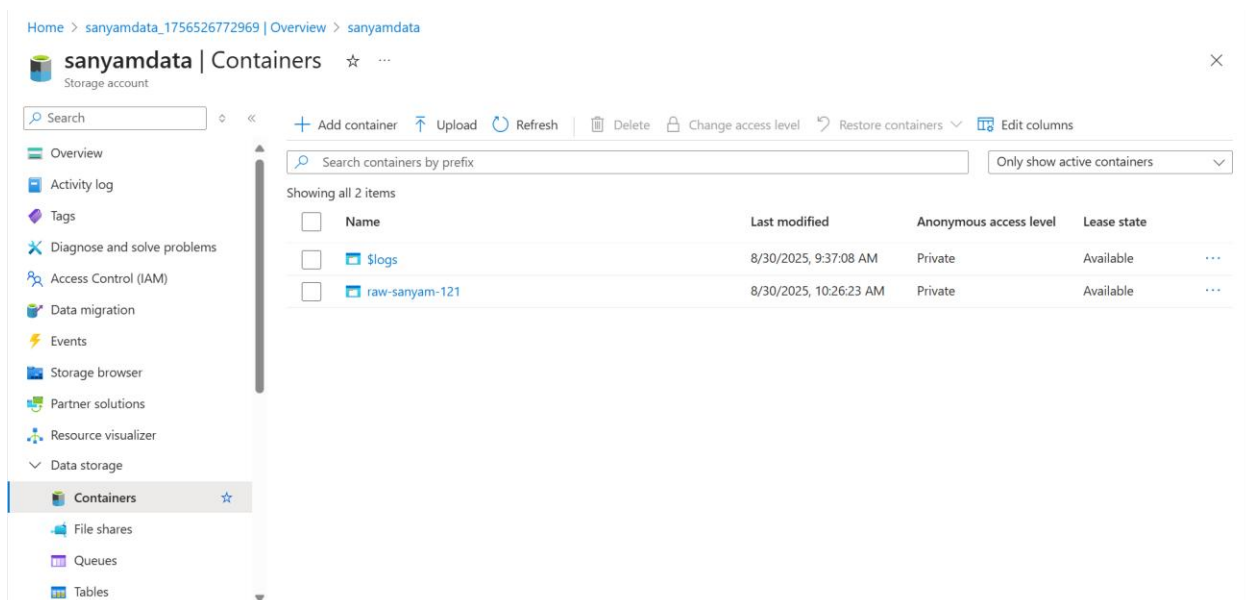
Next steps

[Go to resource](#)

Give feedback

[Tell us about your experience with deployment](#)

Step-2: I have added a container named 'raw-sanyam-121' in the storage account, and within it, I created a directories called 'patients', 'patient-doctor-map', 'labs', 'vitals', and 'visits'. Inside directories, I have ingested a CSV, json and parquet files of all the folders.



Home > **sanyamdata_1756526772969** | Overview > **sanyamdata**

sanyamdata | Containers

Storage account

Search

+ Add container Upload Refresh Delete Change access level Restore containers Edit columns

Search containers by prefix Only show active containers

Showing all 2 items

☐	Name	Last modified	Anonymous access level	Lease state
☐	\$logs	8/30/2025, 9:37:08 AM	Private	Available ...
☐	raw-sanyam-121	8/30/2025, 10:26:23 AM	Private	Available ...

Overview

Activity log

Tags

Diagnose and solve problems

Access Control (IAM)

Data migration

Events

Storage browser

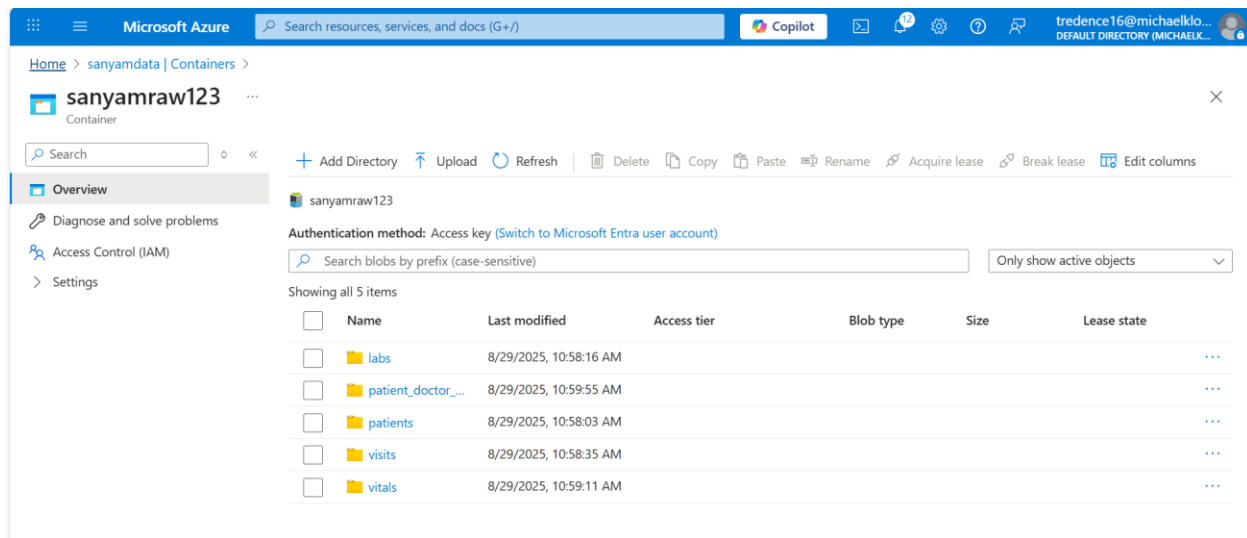
Partner solutions

Resource visualizer

Data storage

Containers

- File shares
- Queues
- Tables



Step 3: I configured IAM permissions on my Azure Storage Account (sanyamdata) by creating Databricks connector and assigning the Storage Blob Data Contributor role to the managed identity used by Databricks.

Purpose: I assigned the Storage Blob Data Contributor role to Databricks' managed identity to ensure secure and authorized access to my data lake. This is a critical step for enabling Delta Live Tables to read, write, and manage data across the Bronze, Silver, and Gold layers in the medallion architecture, while maintaining compliance with Unity Catalog's access and governance requirements.

[Home](#) > [Access Connector for Azure Databricks](#) >

Create an Access Connector for Azure Databricks

Basics Tags Managed Identity Review + create

The Azure Databricks Access Connector lets you connect managed identities to an Azure Databricks account for the purpose of accessing data registered in Unity Catalog.

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

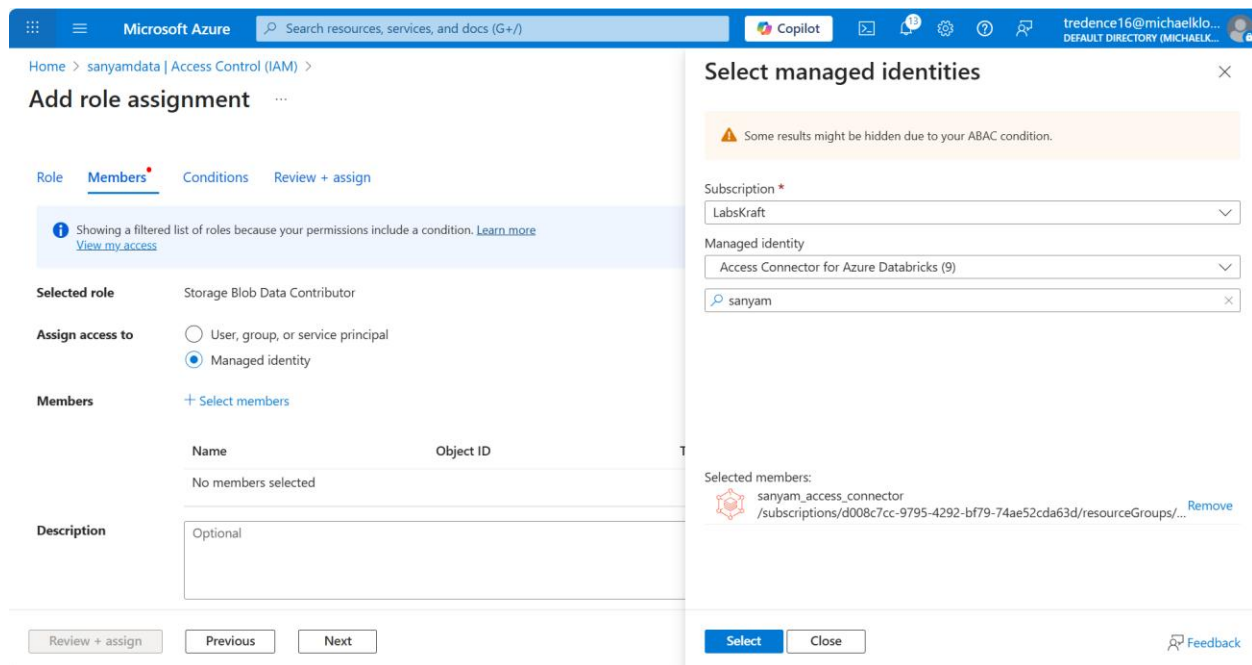
Subscription * ⓘ

Resource group * ⓘ [Create new](#)

Instance details

Name * ⓘ ✓

Region * ⓘ



Step-4: I created a Unity Catalog named “Sanyam_capstone2_catalog” to manage data governance across my Databricks workspace. Before creating the catalog, I set up an external location named “Sanyam_healthcare_ext_loc”, which points to my Azure Data Lake Storage Gen2 (sanyamdata).

Purpose: I created an external location before setting up the Unity Catalog to securely link my Azure Data Lake Storage Gen2 with Databricks. This step is essential for enabling Unity Catalog to manage access, enforce governance, and organize data across the Bronze, Silver, and Gold layers of the medallion architecture. The external location acts as the base path for organizing your Bronze, Silver, and Gold layers.

It enables:

- Structured data management.
- Schema enforcement and lineage tracking.
- Integration with Auto Loader and Delta Live Tables.



Test connection

Overview Permissions Browse Workspaces

Description

Add description

About this external location

Owner Tredence LabsKraft16

Fallback mode: ☐

Credential	sanyam_healthcare_ext_loc_azuremanagedidentity_1756530550648
URL	abfss://raw-sanyam-121@sanyamdata.dfs.core.windows.net/
Limit to read-only use	Disabled

Catalog

● kraft100 8 GB, 4 Cores

Type to search...



- My organization
 - tred_d5_4405446970161177
 - system
 - healthcare_sanyam_catalog
 - bronze
 - default
 - gold
 - information_schema
 - silver
 - sanyam_capstone3_catalog
 - shiv_catalog
 - cap1_shiv_catalog
 - cap2_shiv_catalog
 - ddcatalog07
 - healthcare_vaishnavi
 - nilay_healthcare_catalog
 - shibang



Share ▾

Create schema

Overview Details Permissions Workspaces

Description

AI generate Add

Filter schemas

5 schemas

Name	Owner	Created at
bronze	tredence16@michaelkloudkraf...	Aug 30, 2025, 04:56 PM
default	tredence16@michaelkloudkraf...	Aug 30, 2025, 10:41 AM
gold	tredence16@michaelkloudkraf...	Aug 30, 2025, 05:46 PM
information_schema	System user	Aug 30, 2025, 10:41 AM
silver	tredence16@michaelkloudkraf...	Aug 30, 2025, 05:32 PM

About this catalog

Owner Tredence LabsKraft16

BRONZE LAYER CODE:

```
import json from pyspark.sql.types

import *from pyspark.sql.functions import *

catalog = "healthcare_sanyam_catalog"

schema = "bronze"

spark.sql(f"USE CATALOG {catalog}")

spark.sql(f"CREATE SCHEMA IF NOT EXISTS {schema}")
```

```
spark.sql(f"USE {schema}")
```

```
raw_labs_data = "abfss://raw-sanyam-121@sanyamdata.dfs.core.windows.net/labs"
```

```
raw_patients_data = "abfss://raw-sanyam-121@sanyamdata.dfs.core.windows.net/patients"
```

```
raw_visits_data = "abfss://raw-sanyam-121@sanyamdata.dfs.core.windows.net/visits"
```

```
raw_vitals_data = "abfss://raw-sanyam-121@sanyamdata.dfs.core.windows.net/vitals"
```

```
raw_patient_doctor = "abfss://raw-sanyam-121@sanyamdata.dfs.core.windows.net/patients_doctor_map"
```

-----CREATION OF TABLES-----

```
%sql
```

```
CREATE TABLE IF NOT EXISTS bronze.bronze_patients
```

```
USING CSV
```

```
OPTIONS (
```

```
  path 'abfss://raw-sanyam-121@sanyamdata.dfs.core.windows.net/patients/patients.csv',
```

```
  header 'true',
```

```
  inferSchema 'true'
```

```
);
```

```
%sql
```

```
CREATE TABLE IF NOT EXISTS bronze.bronze_labs
```

```
USING CSV
```

```
OPTIONS (
```

```
  path 'abfss://raw-sanyam-121@sanyamdata.dfs.core.windows.net/labs/labs.csv',
```

```
  header 'true',
```

```
  inferSchema 'true'
```

```
);
```

```
%sql
```

```
CREATE TABLE IF NOT EXISTS bronze.bronze_vitals
```

```
USING CSV
```

```
OPTIONS (
```

```
  path 'abfss://raw-sanyam-121@sanyamdata.dfs.core.windows.net/vitals/vitals.csv',
```

```
  header 'true',
```

```
  inferSchema 'true'
```

```
);
```

```
%sql
```

```
CREATE TABLE IF NOT EXISTS bronze.bronze_visits
```

```
USING CSV
```

```
OPTIONS (
```

```
  path 'abfss://raw-sanyam-121@sanyamdata.dfs.core.windows.net/visits/visits.csv',
```

```
  header 'true',
```

```
  inferSchema 'true'
```

```
);
```

```
%sql
```

```
CREATE TABLE IF NOT EXISTS bronze.bronze_patients_doctor_map
```

```
USING CSV
```

```
OPTIONS (
```

```
  path 'abfss://raw-sanyam-121@sanyamdata.dfs.core.windows.net/patients_doctor_map/patient_doctor_map.csv',
```

```
  header 'true',
```

```
  inferSchema 'true'
```

```
);
```

▶ 02:31 PM (2s) 5			
%sql show tables;			
▶ _sqldf: pyspark.sql.connect.dataframe.DataFrame = [database: string, tableName: string ... 1 more field]			
Table ▾	+		
	database	tableName	isTemporary
1	bronze	bronze_labs	false
2	bronze	bronze_patients	false
3	bronze	bronze_patients_doctor_m...	false
4	bronze	bronze_visits	false
5	bronze	bronze_vitals	false
6		_sqldf	true

SILVER LAYER CODE

Patients file clean code:

```
from pyspark.sql.functions import col, when

patients_df = spark.table("healthcare_sanyam_catalog.bronze.bronze_patients")

patients_df_cleaned = patients_df.select([

    when(col(c).rlike("INVALID_*"), "Unknown").otherwise(col(c)).alias(c)

    for c in patients_df.columns

])

patients_df_cleaned = patients_df_cleaned.withColumn("age", col("age").cast("int"))

patients_df_cleaned = patients_df_cleaned.filter((col("age").isNotNull()) & (col("age") <= 120))

patients_df_cleaned = patients_df_cleaned.withColumn("gender",

    when(col("gender").isin("M", "F", "Other"), col("gender")).otherwise("Other")

)

patients_df_cleaned = patients_df_cleaned.fillna({

    "age": 50.0,

    "gender": "Other",

    "diagnosis": "Undiagnosed",

    "prescription": "None"

})

patients_df_cleaned = patients_df_cleaned.dropna(subset=["patient_id", "name"])

patients_df_cleaned = patients_df_cleaned.dropDuplicates()

patients_df_cleaned = patients_df_cleaned.filter(col("prescription") != "Unknown")

patients_df_cleaned = patients_df_cleaned.select(

    col("patient_id").cast("string"),

    col("name").cast("string"),
```

```
patients_df_cleaned.write \
```

 173 rows | 8.56s runtime

Patient_doctor_map Code

```
from pyspark.sql.functions import col, when

doctor_map_df =
spark.table("healthcare_sanyam_catalog.bronze.bronze_patients_doctor_map")

doctor_map_cleaned = doctor_map_df.select([

    when(col(c).rlike("INVALID_*"), "Unknown").otherwise(col(c)).alias(c)

    for c in doctor_map_df.columns

])

doctor_map_cleaned = doctor_map_cleaned.dropna(subset=["patient_id", "doctor_id"])

doctor_map_cleaned = doctor_map_cleaned.fillna({"care_team": "Unassigned"})

doctor_map_cleaned = doctor_map_cleaned.filter(col("care_team") != "Unknown")

doctor_map_cleaned = doctor_map_cleaned.filter(col("doctor_id") != "Unknown")

doctor_map_cleaned = doctor_map_cleaned.dropDuplicates()

doctor_map_cleaned = doctor_map_cleaned.select(

    col("patient_id").cast("string"),

    col("doctor_id").cast("string"),

    col("care_team").cast("string")

)

doctor_map_cleaned.write \

    .format("delta") \

    .mode("overwrite") \

    .saveAsTable("healthcare_sanyam_catalog.silver.patient_doctor_map_silver")
```

► (1) Spark Jobs

►  _sqldf: pyspark.sql.connect.dataframe.DataFrame = [patient_id: string, doctor_id: string ... 1 more field]

Table ▼

+

	^A _C patient_id	^A _C doctor_id	^A _C care_team
1	P00112	D004	Cardio
2	P00074	D002	Endocrine
3	P00143	D003	Cardio
4	P00190	D003	Cardio
5	P00051	D002	General
6	P00085	D001	General
7	P00133	D004	Endocrine
8	P00186	D004	Endocrine
9	P00091	D005	Endocrine
10	P00018	D004	Cardio
11	P00059	D005	Pulmonary
12	P00029	D002	Unassigned
13	P00134	D003	General
14	P00170	D002	Cardio
15	P00097	D004	General

Labs_clean Code

```
from pyspark.sql.functions import col, when

labs_df = spark.table("healthcare_sanyam_catalog.bronze.bronze_labs")

labs_cleaned = labs_df.select([

    when(col(c).rlike("^INVALID_*"), "Unknown").otherwise(col(c)).alias(c)

    for c in labs_df.columns

])

labs_cleaned = labs_cleaned.dropna(subset=["patient_id", "test_name"])

labs_cleaned = labs_cleaned.fillna({"unit": "Unspecified", "date": "Unspecified"})

labs_cleaned = labs_cleaned.withColumn("result", col("result").cast("float"))

labs_cleaned = labs_cleaned.filter((col("result").isNotNull()) & (col("result") >= 0) & (col("result") <= 1000))
```

```
labs_cleaned = labs_cleaned.dropDuplicates()

labs_cleaned = labs_cleaned.filter(col("date") != "Unknown")

labs_cleaned = labs_cleaned.filter(col("date") != "Unspecified")

labs_cleaned = labs_cleaned.select(

    col("patient_id").cast("string"),

    col("test_name").cast("string"),

    col("result").cast("float"),

    col("unit").cast("string"),

    col("date").cast("string")

)

labs_cleaned.write \

    .mode("overwrite") \

    .format("delta") \

    .saveAsTable("healthcare_sanyam_catalog.silver.labs_silver")
```

Table ▾

+

🔍

🔼

📄

🗒

	A ^B C patient_id	A ^B C test_name	1.2 result	A ^B C unit	A ^B C date
1	P00135	COVID19 PCR	4.239999771118164	%	2025-07-22
2	P00057	HbA1c	6.289999961853027	x10^9/L	2025-08-29
3	P00187	CBC	14.4499998092651...	%	2025-08-05
4	P00168	ECG	4.019999980926514	mg/dL	2025-08-28
5	P00148	HbA1c	8.350000381469727	mg/dL	2025-08-21
6	P00098	COVID19 PCR	3.09999990463256...	%	2025-08-02
7	P00170	HbA1c	11.8000001907348...	%	2025-08-16
8	P00025	HbA1c	1.87999999523162...	%	2025-07-13
9	P00140	ECG	9.739999771118164	mg/dL	2025-07-14
10	P00006	HbA1c	4.760000228881836	%	2025-08-21
11	P00136	Lipid Panel	13.3800001144409...	mg/dL	2025-07-02
12	P00150	ECG	8.239999771118164	mg/dL	2025-07-21
13	P00141	HbA1c	9.270000457763672	%	2025-07-06
14	P00137	COVID19 PCR	0.91000002622604...	mg/dL	2025-07-21
15	P00096	ECG	12.8100004196167	mg/dL	2025-08-12

↓

859 rows | 9.23s runtime

Refreshed 1 hour ago

Visits_clean Code

```
from pyspark.sql.functions import col, when

visits_df = spark.table("healthcare_sanyam_catalog.bronze.bronze_visits")

visits_cleaned = visits_df.select([

    when(col(c).rlike("^INVALID_.*"), "Unknown").otherwise(col(c)).alias(c)

    for c in visits_df.columns

])

visits_cleaned = visits_cleaned.dropna(subset=["patient_id", "admission_date"])

visits_cleaned = visits_cleaned.fillna({"discharge_date": "Unspecified", "reason": "Unspecified"})

visits_cleaned = visits_cleaned.filter(col("admission_date") != "Unknown")

visits_cleaned = visits_cleaned.filter(col("discharge_date") != "Unknown")

visits_cleaned = visits_cleaned.filter(col("reason") != "Unknown")

visits_cleaned = visits_cleaned.dropDuplicates()

visits_cleaned = visits_cleaned.select(

    col("patient_id").cast("string"),

    col("admission_date").cast("string"),

    col("discharge_date").cast("string"),

    col("reason").cast("string")

)

visits_cleaned.write \

    .mode("overwrite") \

    .format("delta") \

    .saveAsTable("healthcare_sanyam_catalog.silver.visits_clean")
```

Table ▾

+

	^A _C patient_id	^A _C admission_date	^A _C discharge_date	^A _C reason
1	P00167	2025-07-01	2025-07-10	Surgery
2	P00199	2025-04-06	2025-04-16	Surgery
3	P00196	2025-06-29	2025-07-01	Checkup
4	P00060	2025-07-26	2025-07-30	Checkup
5	P00151	2025-06-17	2025-06-18	Surgery
6	P00120	2025-07-14	2025-07-17	Checkup
7	P00186	2025-06-15	2025-06-21	Emergency
8	P00140	2025-03-02	2025-03-07	Checkup
9	P00131	2025-05-14	2025-05-15	Emergency
10	P00147	2025-07-19	2025-07-24	Therapy
11	P00087	2025-08-11	2025-08-13	Surgery
12	P00193	2025-07-04	2025-07-08	Surgery

Vitals_silver Code

```
from pyspark.sql.functions import col, when, to_timestamp
```

```
df = spark.table("healthcare_sanyam_catalog.bronze.bronze_vitals")
```

```
df_cleaned = df.select([
```

```
    when(col(c).rlike("^INVALID_*"), "Unknown").otherwise(col(c)).alias(c)
```

```
    for c in df.columns
```

```
])
```

```
df_cleaned = df_cleaned.dropna(subset=["patient_id", "timestamp"])
```

```
df_cleaned = df_cleaned.fillna({
```

```
    "device_id": "Unspecified",
```

```
    "vital_type": "Unspecified",
```

```
    "vital_value": "Unspecified"
```

```
})
```

```
df_cleaned = df_cleaned.withColumn("timestamp", to_timestamp(col("timestamp"), "yyyy-MM-dd'T'HH:mm:ss.SSSSSS"))
```

```

df_cleaned = df_cleaned.filter(col("timestamp").isNotNull())

# Cast vital_value to float and filter out unrealistic values

df_cleaned = df_cleaned.withColumn("vital_value", col("vital_value").cast("float"))

df_cleaned = df_cleaned.filter((col("vital_value").isNotNull()) & (col("vital_value") >= 0) & (col("vital_value") <=
1000))

df_cleaned = df_cleaned.filter(col("vital_type") != "Unspecified")

df_cleaned = df_cleaned.filter(col("device_id") != "Unspecified")

df_cleaned = df_cleaned.dropDuplicates()

df_cleaned = df_cleaned.select(
    col("patient_id").cast("string"),
    col("device_id").cast("string"),
    col("timestamp"),
    col("vital_type").cast("string"),
    col("vital_value").cast("float")
)

df_cleaned.write \

    .format("delta") \

    .mode("overwrite") \

    .option("overwriteSchema", "true") \

    .saveAsTable("healthcare_sanyam_catalog.silver.vitals_silver")

```

	patient_id	device_id	timestamp	vital_type	vital_value
1	P00179	DEV_R4WMP	2025-08-29T09:09:06.046+00:00	oxygen_saturation	99.4000015258789
2	P00117	DEV_UVW1E	2025-08-28T22:45:17.046+00:00	oxygen_saturation	99.80000305175781
3	P00186	DEV_ZBW03	2025-08-28T23:48:39.046+00:00	systolic_bp	151.600006103515...
4	P00050	DEV_6FAZR	2025-08-28T19:29:12.046+00:00	glucose	78.5
5	P00063	DEV_L61KM	2025-08-28T15:17:30.046+00:00	glucose	74.4000015258789
6	P00178	DEV_J18GF	2025-08-28T10:49:58.046+00:00	heart_rate	125.5999984741211
7	P00130	DEV_H2XQ5	2025-08-28T21:49:04.046+00:00	glucose	84
8	P00189	DEV_MFZE0	2025-08-29T02:21:38.046+00:00	systolic_bp	125.5999984741211
9	P00092	DEV_ML469	2025-08-29T00:11:55.046+00:00	heart_rate	116.0999984741211
10	P00193	DEV_KMSQN	2025-08-28T22:54:49.046+00:00	systolic_bp	112.800003051757...
11	P00191	DEV_2FUNV	2025-08-29T08:00:33.046+00:00	Unknown	91.80000305175781
12	P00021	DEV_6NGPA	2025-08-29T00:54:15.046+00:00	systolic_bp	149.6999969482422
13	P00062	DEV_GBA4H	2025-08-29T01:53:50.046+00:00	glucose	134.899993896484...

Gold layer code:

```
from pyspark.sql.functions import col, to_timestamp, avg, max, datediff

patients = spark.table("healthcare_sanyam_catalog.sanyam_healthcare_db.patients_clean")

visits = spark.table("healthcare_sanyam_catalog.sanyam_healthcare_db.visits_clean")

labs = spark.table("healthcare_sanyam_catalog.sanyam_healthcare_db.labs_clean")

vitals = spark.table("healthcare_sanyam_catalog.sanyam_healthcare_db.vitals_clean")

map_df = spark.table("healthcare_sanyam_catalog.sanyam_healthcare_db.patient_doctor_map_clean")

visits = visits.withColumn("visit_duration", datediff(col("discharge_date"), col("admission_date")))

lab_summary = labs.filter(col("test_name") == "ECG") \

    .groupBy("patient_id") \

    .agg(avg("result").alias("average_glucose"))

vital_summary = vitals.groupBy("patient_id") \

    .agg(max("timestamp").alias("latest_vital_timestamp"))

gold_df = patients.join(map_df, "patient_id", "left") \

    .join(visits, "patient_id", "left") \

    .join(lab_summary, "patient_id", "left") \

    .join(vital_summary, "patient_id", "left")

gold_df.write \

    .format("delta") \

    .mode("overwrite") \

    .option("overwriteSchema", "true") \

    .saveAsTable("healthcare_sanyam_catalog.sanyam_healthcare_db.gold_patient_summary")
```

-
- ▶  gold_df: pyspark.sql.connect.dataframe.DataFrame = [patient_id: string, name: string ... 12 more fields]
 - ▶  lab_summary: pyspark.sql.connect.dataframe.DataFrame = [patient_id: string, average_glucose: double]
 - ▶  labs: pyspark.sql.connect.dataframe.DataFrame = [patient_id: string, test_name: string ... 3 more fields]
 - ▶  map_df: pyspark.sql.connect.dataframe.DataFrame = [patient_id: string, doctor_id: string ... 1 more field]
 - ▶  patients: pyspark.sql.connect.dataframe.DataFrame = [patient_id: string, name: string ... 4 more fields]
 - ▶  visits: pyspark.sql.connect.dataframe.DataFrame = [patient_id: string, admission_date: date ... 3 more fields]
 - ▶  vital_summary: pyspark.sql.connect.dataframe.DataFrame = [patient_id: string, latest_vital_timestamp: timestamp]
 - ▶  vitals: pyspark.sql.connect.dataframe.DataFrame = [patient_id: string, device_id: string ... 3 more fields]
-

TIME TRAVEL

For Gold layer:

```
from delta.tables import DeltaTable
```

```
delta_table = DeltaTable.forName(spark, "healthcare_sanyam_catalog.gold.gold_patient_summary")
```

```
history_df = delta_table.history()
```

```
history_df.show(truncate=False)
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|1|2025-08-31 09:10:24|145412978324136|tredence16@michaelkloudkrafthotmail.onmicrosoft.com|CREATE OR REPLACE TABLE
AS SELECT[{partitionBy -> [], clusterBy -> [], description -> NULL, isManaged -> true, properties -> {"delta.enableDeleti
onVectors":"true"}, statsOnLoad -> false}|NULL|{103245433696827}|0831-034916-a97pi9bf|0|WriteSerializable|fals
e|{numFiles -> 1, numRemovedFiles -> 1, numRemovedBytes -> 15221, numOutputRows -> 970, numOutputBytes -> 14830}|
NULL|Databricks-Runtime/16.4.x-scala2.12|
|0|2025-08-30 12:19:51|145412978324136|tredence16@michaelkloudkrafthotmail.onmicrosoft.com|CREATE OR REPLACE TABLE
AS SELECT[{partitionBy -> [], clusterBy -> [], description -> NULL, isManaged -> true, properties -> {"delta.enableDeleti
onVectors":"true"}, statsOnLoad -> false}|NULL|{3869432229168326}|0830-034604-5azacsr|NULL|WriteSerializable|fals
e|{numFiles -> 1, numRemovedFiles -> 0, numRemovedBytes -> 0, numOutputRows -> 970, numOutputBytes -> 15221}|
NULL|Databricks-Runtime/16.4.x-scala2.12|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

```
df_version = spark.read.format("delta") \
```

```
.option("versionAsOf", 1) \
```

```
.table("healthcare_sanyam_catalog.gold.gold_patient_summary")
```

```
df_version.show()
```

y	P00020 Patient_U4WK	44	Other	Diabetes	DrugA	D005 Endocrine	2025-03-22	2025-03-29	Emergenc
	7		NULL	2025-08-29 06:12:...					
y	P00020 Patient_U4WK	44	Other	Diabetes	DrugA	D005 Endocrine	2025-05-29	2025-06-01	Emergenc
	3		NULL	2025-08-29 06:12:...					
y	P00142 Patient_EQSO	24	F	Diabetes	DrugB	D003 General	2025-05-28	2025-06-05	Surger
	8		NULL	2025-08-29 08:18:...					
y	P00142 Patient_EQSO	24	F	Diabetes	DrugB	D003 General	2025-05-29	2025-06-02	Emergenc
	4		NULL	2025-08-29 08:18:...					
y	P00077 Patient_X0A1	23	Other Heart Disease	None	D001 Pulmonary	2025-05-13	2025-05-18	Therap	
	5		NULL	2025-08-29 07:30:...					
y	P00077 Patient_X0A1	23	Other Heart Disease	None	D001 Pulmonary	2025-06-21	2025-06-25	Emergenc	
	4		NULL	2025-08-29 07:30:...					
p	P00177 Patient_KVRG	83	Other Heart Disease	DrugA	D004 Pulmonary	2025-03-11	2025-03-13	Checku	
	2		NULL	2025-08-29 08:56:...					
y	P00177 Patient_KVRG	83	Other Heart Disease	DrugA	D004 Pulmonary	2025-03-23	2025-04-02	Therap	
	10		NULL	2025-08-29 08:56:...					
p	P00177 Patient_KVRG	83	Other Heart Disease	DrugA	D004 Pulmonary	2025-04-02	2025-04-07	Checku	
	5		NULL	2025-08-29 08:56:...					

In terms of Timestamp:

```
df_time = spark.read.format("delta") \

.option("timestampAsOf", "2025-08-30T12:19:51") \

.table("healthcare_sanyam_catalog.gold.gold_patient_summary")

df_time.show()
```

df_time: pyspark.sql.connect.dataframe.DataFrame = [patient_id: string, name: string ... 12 more fields]

patient_id	name	age	gender	diagnosis	prescription	doctor_id	care_team	admission_date	discharge_date	reason
P00024	Patient_R0L9	27	F	None	DrugC	D001	General	2025-05-29	2025-06-04	Surgery
P00024	Patient_R0L9	27	F	None	DrugC	D001	General	2025-07-02	2025-07-11	Checkup
P00024	Patient_R0L9	27	F	None	DrugC	D001	General	2025-06-04	2025-06-10	Surgery
P00024	Patient_R0L9	27	F	None	DrugC	D001	General	2025-08-10	2025-08-15	Emergency
P00024	Patient_R0L9	27	F	None	DrugC	D001	General	2025-06-10	2025-06-12	Therapy
P00024	Patient_R0L9	27	F	None	DrugC	D001	General	2025-06-11	2025-06-18	Checkup
P00024	Patient_R0L9	27	F	None	DrugC	D001	General	2025-06-02	Unspecified	Therapy
P00134	Patient_GTCB	41	Other	Heart Disease	DrugC	D003	General	2025-04-03	2025-04-13	Emergency

RETENTION POLICY

```
%sql

SET spark.databricks.delta.retentionDurationCheck.enabled = false;

ALTER TABLE healthcare_sanyam_catalog.silver.patients_silver SET TBLPROPERTIES(

'delta.logRetentionDuration'= '3650 days',

'delta.deletedFileRetentionDuration' = '3650 days'

);
```

Purpose of RETENTION POLICY:

- Automatically deletes old data to **reduce storage costs**.
- Ensures data is retained for a **minimum duration** to meet legal or regulatory requirements (e.g., HIPAA, GDPR).
- Reduces the size of active datasets, improving **query performance** and **streaming efficiency**.

MASKING OF PATIENT'S DATA (PII)

Code:

```
CREATE OR REPLACE VIEW healthcare_sanyam_catalog.silver.patients_masked AS
```

```
SELECT
```

```
  CASE
```

```
    WHEN is_member('PII_Members') THEN name
```

```
    ELSE CONCAT(
```

```
      LEFT(name, 1),
```

```
      REPEAT('*', LENGTH(name) - 2),
```

```
      RIGHT(name, 1)
```

```
    )
```

```
  END AS name,
```

```
  age,
```

```
  gender,
```

```
  diagnosis,
```

```
  prescription,
```

```
  patient_id
```

```
FROM healthcare_sanyam_catalog.silver.patients_silver;
```

OUTPUT:

Table						
	A ^B _C name	1 ² ₃ age	A ^B _C gender	A ^B _C diagnosis	A ^B _C prescription	A ^B _C patient_id
1	p*****G	27	F	None	DrugC	P00024
2	p*****B	41	Other	Heart Disease	DrugC	P00134
3	p*****K	44	Other	Diabetes	DrugA	P00020
4	p*****O	24	F	Diabetes	DrugB	P00142
5	p*****I	23	Other	Heart Disease	None	P00077
6	p*****G	83	Other	Heart Disease	DrugA	P00177
7	p*****B	32	M	None	DrugB	P00162
8	p*****M	53	Other	Diabetes	DrugC	P00099
9	p*****G	60	Other	Heart Disease	None	P00045
10	p*****F	20	Other	Hypertension	None	P00161
11	p*****M	69	Other	Heart Disease	DrugA	P00070
12	p*****O	84	M	Diabetes	DrugC	P00095
13	p*****F	38	Other	Hypertension	DrugB	P00104

ROLE BASED ACCESS CONTROL:

Settings

Workspace admin

Appearance

Identity and access

Security

Compute

Development

Notifications

Advanced

User

Profile

Preferences

Developer

Linked accounts

Notifications

Identity and access

Management and permissions

Users

Manage users and entitlements

Manage

Groups

Manage groups and entitlements

Manage

Service principals

Manage service principals and entitlements

Manage

Workspace settings > Identity and access > Groups >

Group details

PII_Members

Delete

Group Information

Members

Entitlements

Permissions

Parent groups

i

Permissions control who can manage this group. [Learn more](#)

x

Q

Search principals

Grant access

Principal	Permissions
Tredence LabsKraft16 (tredence16@michaelkloudkrafthot...	Group: Manage

Catalog

kraft100 8 GB, 4 Cores

Type to search...

My organization

tred_d5_4405446970161177

system

healthcare_sanyam_catalog

bronze

default

gold

information_schema

silver

sanyam_capstone3_catalog

default

information_schema

sanyam_capstone3_db

shiv_catalog

cap1_shiv_catalog

cap2_shiv_catalog

ddcatalog07

healthcare_vaishnavi

healthcare_sanyam_catalog

Share

Create schema

OverviewDetailsPermissionsWorkspaces

Grant

Revoke

Privileges

Type to filter by principal

Principal	Privilege	Object
asanyam10@gmail.com	EXECUTE	healthcare_sanyam_catalog
asanyam10@gmail.com	EXTERNAL USE SCHEMA	healthcare_sanyam_catalog
asanyam10@gmail.com	READ VOLUME	healthcare_sanyam_catalog
asanyam10@gmail.com	SELECT	healthcare_sanyam_catalog
asanyam10@gmail.com	USE CATALOG	healthcare_sanyam_catalog
asanyam10@gmail.com	USE SCHEMA	healthcare_sanyam_catalog
asanyam21@gmail.com	EXECUTE	healthcare_sanyam_catalog
asanyam21@gmail.com	EXTERNAL USE SCHEMA	healthcare_sanyam_catalog
asanyam21@gmail.com	READ VOLUME	healthcare_sanyam_catalog
asanyam21@gmail.com	SELECT	healthcare_sanyam_catalog
asanyam21@gmail.com	USE CATALOG	healthcare_sanyam_catalog
asanyam21@gmail.com	USE SCHEMA	healthcare_sanyam_catalog
All account users	BROWSE	healthcare_sanyam_catalog

I have given the access to users for read, edit and create. I have given the read access to ‘asanyam10@gmail.com’ and asanyam21@gmail.com’.

AZURE -EVENTHUB Streaming Ingestion:

[Home](#) > [Event Hubs](#) >

Create Namespace

Event Hubs

Project Details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

Resource group * [Create new](#)

Instance Details

Enter required settings for this namespace, including a price tier and configuring the number of units (capacity).

Namespace name * [.servicebus.windows.net](#)

Region *
 ⓘ The region selected supports Availability zones. Your namespace will have Availability Zones enabled. [Learn more.](#)

Pricing tier *
 [Browse the available plans and their features](#)

CODE 1:

```
# Event Hub connection parameters
```

```
eh_namespace = "sanyam-eventhub"
```

```
eh_name = "sanyam-event"
```

```
eh_sas_key_name = "RootManageSharedAccessKey"
```

```
eh_sas_key = "SMN56vGHQ+JghHJi7ww3+HIJKoNu+eAHLJ9Tno="
```

```
connection_string =
```

```
f"Endpoint=sb://{eh_namespace}.servicebus.windows.net/;SharedAccessKeyName={eh_sas_key_name};SharedAccessKey={eh_sas_key};EntityPath={eh_name}"
```

CODE 2:

```
from pyspark.sql.types import StructType, StructField, StringType, DoubleType
```

```
from pyspark.sql import functions as F
```

```
# Define schema for patient messages
```

```
patient_schema = StructType([
```

```
    StructField("patient_id", StringType(), True),
```

```

    StructField("name", StringType(), True),
    StructField("age", DoubleType(), True),
    StructField("gender", StringType(), True),
    StructField("diagnosis", StringType(), True),
    StructField("prescription", StringType(), True)
])

# Read streaming data from Event Hub

stream_df = (spark.readStream

    .format("eventhubs")

    .option("eventhubs.connectionString", connection_string)

    .load()

)

# Decode Event Hub messages from binary to string and parse JSON

decoded_df = (stream_df

    .withColumn("body_str", F.col("body").cast("string"))

    .select(F.from_json("body_str", patient_schema).alias("data"))

    .select("data.*")

)

# Write stream to Bronze Delta table

bronze_stream = (decoded_df.writeStream

    .format("delta")

    .outputMode("append")

    .option("checkpointLocation", "/mnt/delta/patients_bronze/_checkpoints/")

    .table("healthcare_sanyam_catalog.bronze.patients_bronze")

)

```

Output:

```

❗ > [UNSUPPORTED_STREAMING_SOURCE_PERMISSION_ENFORCED] Data source eventhubs is not supported as a streaming source on
a shared cluster. SQLSTATE: 0A000

```

Installing MAVEN in our systems:

[Catalog Explorer](#) >

metastore_azure_southeastasia



southeastasia

External delta sharing: disabled



System user

Details

Permissions

Allowed JARs/Init Scripts

Account administrators can go to [Account console](#) to edit metastore details.

Metastore ID	72655e9c-2170-440a-a82b-c983b8413366
Region	southeastasia
External delta sharing	Disabled

Install library [Send feedback](#)



Library Source ⓘ



Workspace



Volumes



File Path/ADLS



PyPI



Maven

Coordinates

com.microsoft.azure:azure-eventhubs-spark_2.12:2.3.22

[Search Packages](#)

Repository ⓘ

Optional

Exclusions

Dependencies to exclude (log4j:log4junit:junit)

JARs and Maven libraries on shared clusters must be on the allowlist. Please see the doc about adding JARs and Maven libraries to allowlist <https://docs.databricks.com/en/data-governance/unity-catalog/manage-privileges/allowlist.html>

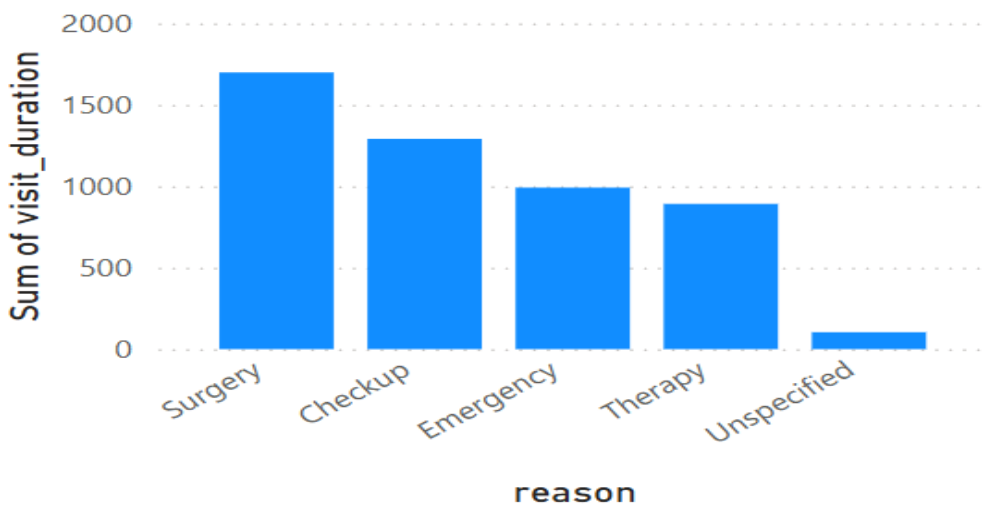
Failed libraries: com.microsoft.azure:azure-eventhubs-spark_2.12:2.3.22: PERMISSION_DENIED:
'com.microsoft.azure:azure-eventhubs-spark_2.12:2.3.22' is not in the artifact allowlist

Cancel

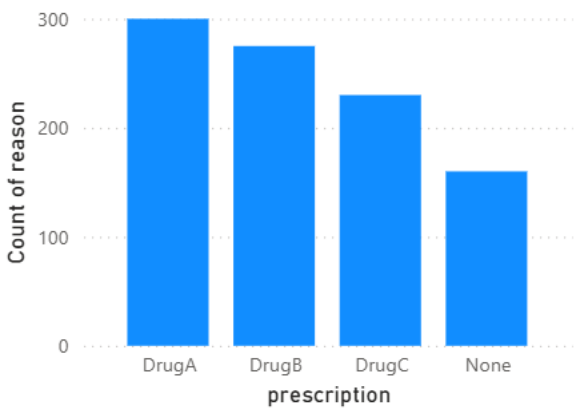
Install

POWER BI OUTPUT

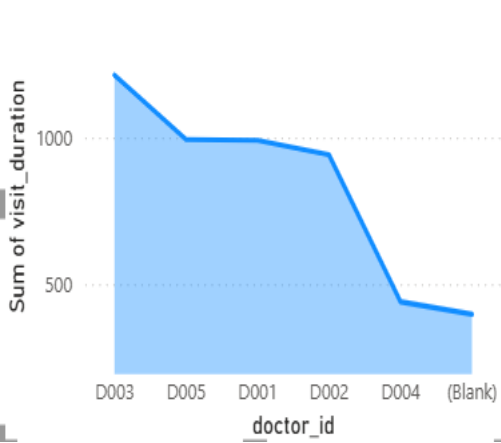
Sum of visit_duration by reason



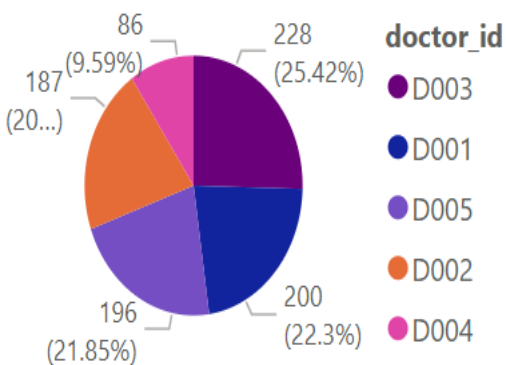
Count of reason by prescription



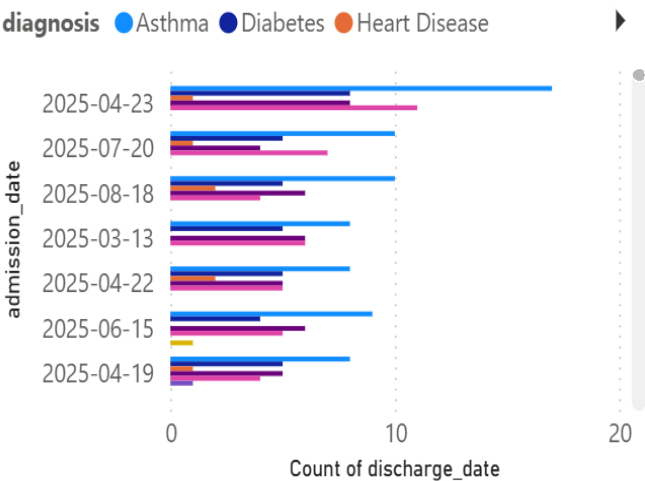
Sum of visit_duration by doctor_id



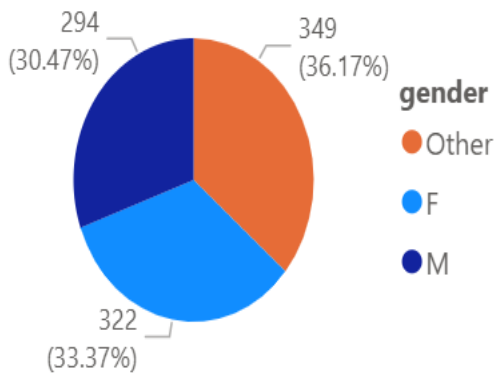
Count of care_team by doctor_id



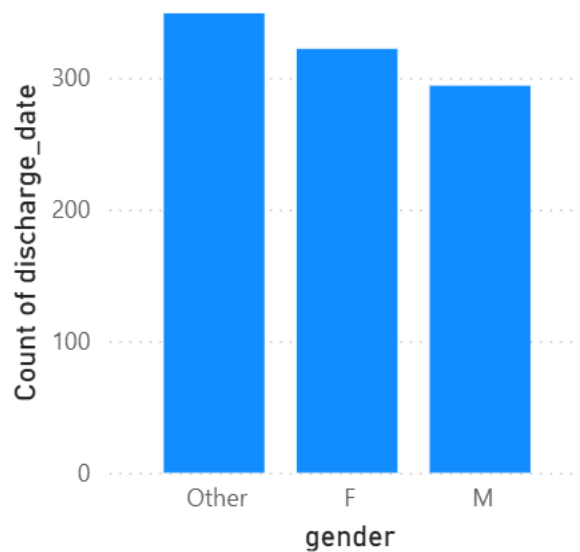
Count of discharge_date by admission_date and diagnosis



Count of admission_date by gender



Count of discharge_date by gender



Count of admission_date by reason and Year

